```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!pip install torch torchvision tqdm flask flask-cors pillow
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cu128)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Collecting flask-cors
  Downloading flask_cors-6.0.2-py3-none-any.whl.metadata (5.3 kB)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.9
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.9
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.9
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->to
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.
Downloading flask_cors-6.0.2-py3-none-any.whl (13 kB)
Installing collected packages: flask-cors
Successfully installed flask-cors-6.0.2
```

```
import os
print(os.listdir('/content/drive/MyDrive/PLANTVILLAGE'))
```

```
['Tomato__Tomato_YellowLeaf__Curl_Virus', 'Tomato__Tomato_mosaic_virus', 'Tomato__Target_Spot', 'Tomato_Septoria_leaf_spot', '
```

```
import os, json, torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights
from torch.optim.lr_scheduler import CosineAnnealingLR
from tqdm import tqdm

# CONFIG
DATA_DIR    = "/content/drive/MyDrive/PLANTVILLAGE"
MODEL_SAVE  = "plant_disease_efficientnet.pth"
CLASS_JSON  = "class_names.json"
IMG_SIZE    = 224
BATCH_SIZE  = 32
EPOCHS      = 10
LR          = 1e-4
WEIGHT_DECAY= 1e-5
VAL_SPLIT   = 0.15
NUM_WORKERS = 2
DEVICE      = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")
```

```
print(f"Using device: {DEVICE}")

# TRANSFORMS
train_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE+32, IMG_SIZE+32)),
    transforms.RandomCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.2),
    transforms.RandomRotation(30),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225]),
])
val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225]),
])

# DATASET
full_dataset = datasets.ImageFolder(DATA_DIR)
class_names  = full_dataset.classes
num_classes  = len(class_names)
print(f"Classes found: {num_classes} → {class_names}")

with open(CLASS_JSON, "w") as f:
    json.dump(class_names, f, indent=2)

val_size   = int(len(full_dataset) * VAL_SPLIT)
train_size = len(full_dataset) - val_size
train_ds, val_ds = random_split(full_dataset, [train_size, val_size])
train_ds.dataset.transform = train_transform
val_ds.dataset.transform   = val_transform

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,  num_workers=NUM_WORKERS, pin_memory=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS, pin_memory=True)

# MODEL
model = efficientnet_b0(weights=EfficientNet_B0_Weights.IMAGENET1K_V1)
in_features = model.classifier[1].in_features
model.classifier = nn.Sequential(
    nn.Dropout(p=0.4, inplace=True),
    nn.Linear(in_features, num_classes),
)
model = model.to(DEVICE)

# TRAINING
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS, eta_min=1e-6)

best_val_acc = 0.0
for epoch in range(1, EPOCHS+1):
    # Train
    model.train()
    t_loss, t_correct, t_total = 0, 0, 0
    for imgs, labels in tqdm(train_loader, desc=f"Epoch {epoch}/{EPOCHS} Train"):
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        out  = model(imgs)
        loss = criterion(out, labels)
        loss.backward()
        optimizer.step()
        t_loss    += loss.item() * imgs.size(0)
        t_correct += (out.argmax(1) == labels).sum().item()
        t_total   += imgs.size(0)

    # Validate
    model.eval()
    v_loss, v_correct, v_total = 0, 0, 0
    with torch.no_grad():
        for imgs, labels in tqdm(val_loader, desc=f"Epoch {epoch}/{EPOCHS} Val  "):
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            out  = model(imgs)
            loss = criterion(out, labels)
            v_loss    += loss.item() * imgs.size(0)
            v_correct += (out.argmax(1) == labels).sum().item()
```

```
            v_total   += imgs.size(0)

    scheduler.step()
    train_acc = t_correct / t_total
    val_acc   = v_correct / v_total
    tag = " ← BEST" if val_acc > best_val_acc else ""
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save({"epoch": epoch, "model_state": model.state_dict(), "num_classes": num_classes}, MODEL_SAVE)

    print(f"Epoch {epoch:02d}/{EPOCHS} | train_acc={train_acc:.4f} | val_acc={val_acc:.4f}{tag}")

print(f"\n✅ Training complete! Best val accuracy: {best_val_acc:.4f}")
```

```
Using device: cuda
Classes found: 15 → ['Pepper__bell___Bacterial_spot', 'Pepper__bell___healthy', 'Potato___Early_blight', 'Potato___Late_blight
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoint
100%|██████████| 20.5M/20.5M [00:00<00:00, 120MB/s]
Epoch 1/10 Train: 100%|██████████| 549/549 [45:47<00:00,  5.01s/it]
Epoch 1/10 Val  : 100%|██████████| 97/97 [08:08<00:00,  5.03s/it]
Epoch 01/10 | train_acc=0.8723 | val_acc=0.9887 ← BEST
Epoch 2/10 Train: 100%|██████████| 549/549 [01:57<00:00,  4.66it/s]
Epoch 2/10 Val  : 100%|██████████| 97/97 [00:18<00:00,  5.38it/s]
Epoch 02/10 | train_acc=0.9825 | val_acc=0.9929 ← BEST
Epoch 3/10 Train: 100%|██████████| 549/549 [01:57<00:00,  4.66it/s]
Epoch 3/10 Val  : 100%|██████████| 97/97 [00:17<00:00,  5.69it/s]
Epoch 03/10 | train_acc=0.9919 | val_acc=0.9948 ← BEST
Epoch 4/10 Train: 100%|██████████| 549/549 [01:57<00:00,  4.68it/s]
Epoch 4/10 Val  : 100%|██████████| 97/97 [00:17<00:00,  5.52it/s]
Epoch 04/10 | train_acc=0.9955 | val_acc=0.9971 ← BEST
Epoch 5/10 Train: 100%|██████████| 549/549 [01:56<00:00,  4.72it/s]
Epoch 5/10 Val  : 100%|██████████| 97/97 [00:17<00:00,  5.42it/s]
Epoch 05/10 | train_acc=0.9969 | val_acc=0.9977 ← BEST
Epoch 6/10 Train: 100%|██████████| 549/549 [01:55<00:00,  4.75it/s]
Epoch 6/10 Val  : 100%|██████████| 97/97 [00:17<00:00,  5.51it/s]
Epoch 06/10 | train_acc=0.9976 | val_acc=0.9977
Epoch 7/10 Train: 100%|██████████| 549/549 [01:55<00:00,  4.77it/s]
Epoch 7/10 Val  : 100%|██████████| 97/97 [00:17<00:00,  5.53it/s]
Epoch 07/10 | train_acc=0.9987 | val_acc=0.9974
Epoch 8/10 Train: 100%|██████████| 549/549 [01:55<00:00,  4.74it/s]
Epoch 8/10 Val  : 100%|██████████| 97/97 [00:18<00:00,  5.28it/s]
Epoch 08/10 | train_acc=0.9981 | val_acc=0.9981 ← BEST
Epoch 9/10 Train: 100%|██████████| 549/549 [01:57<00:00,  4.67it/s]
Epoch 9/10 Val  : 100%|██████████| 97/97 [00:17<00:00,  5.62it/s]
Epoch 09/10 | train_acc=0.9988 | val_acc=0.9974
Epoch 10/10 Train: 100%|██████████| 549/549 [01:58<00:00,  4.64it/s]
Epoch 10/10 Val  : 100%|██████████| 97/97 [00:18<00:00,  5.29it/s]Epoch 10/10 | train_acc=0.9989 | val_acc=0.9981

✅ Training complete! Best val accuracy: 0.9981
```

```
from google.colab import files
files.download('plant_disease_efficientnet.pth')
files.download('class_names.json')
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.